

# Resiliència en el Codi: La Gestió d'Errors en JavaScript

// Manual de supervivència i diagnòstic per a desenvolupadors

[OBJECTE ERROR]

[TRY/CATCH/THROW]

[ERRORS SILENCIOSOS]

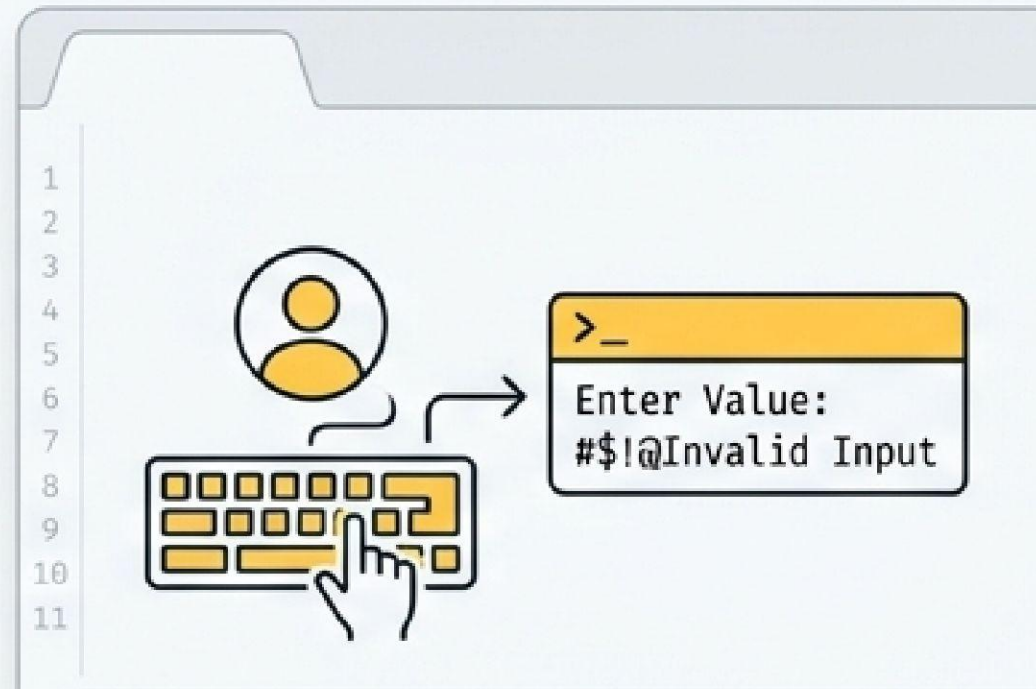


# L'execució sempre falla (i és part del procés)



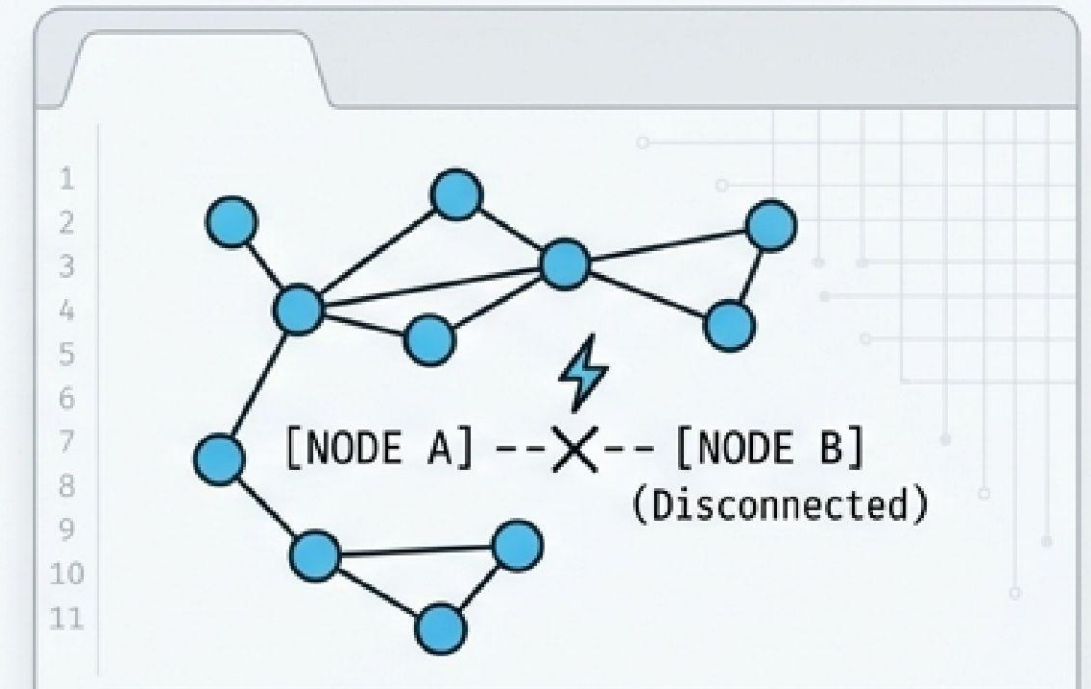
## Errors del Programador

Falles de **sintaxi** o **lògica** introduïdes durant l'escriptura del codi.



## Input Incorrecte

Dades inesperades introduïdes per l'usuari o per una **API externa**.



## Fets Imprevistos

Errors d'entorn, **caigudes de xarxa** o **límits del sistema**.

L'**objectiu no és evitar** que els errors succeeixin, sinó **controlar** com hi reacciona la teva aplicació.



# L'Anatomia de l'Objecte Error

```
1  
2  
3  
4  
5  
6  
7  
8  
9  
10  
11  
const myError = new Error("S'ha produït un error de connexió");
```

La categoria del problema (ex: TypeError).

Constructor natiu per crear instàncies d'error.

La descripció detallada de què ha fallat (ex: 'S'ha produït un error de connexió').

Aquest objecte és el 'pacient' que capturarem i analitzarem durant l'execució.



# Matriu de Diagnòstic: Els tipus d'errors natius

## ReferenceError

**Síntoma:** S'ha utilitzat una variable que no existeix o abans d'inicialitzar-la.

## TypeError

**Síntoma:** El valor és del tipus incorrecte o l'operació no és vàlida per a aquell tipus.

## RangeError

**Síntoma:** Un valor numèric està fora del seu rang permès.

## URIError

**Síntoma:** Ús de caràcters il·legals en funcions de codificació d'URI.

## SyntaxError

**Síntoma:** El codi viola les regles gramaticals de JavaScript.



# Diagnòstic Clínic: Referències i Tipus

## ReferenceError

```
1  
2  
3  
4 let x = y + 1; // y no està definida  
5  
6  
7  
8  
9  
10  
11
```

✖ **ReferenceError:** y is not defined

## TypeError

```
1  
2  
3  
4 let num = 1;  
5  
6 num.toUpperCase();  
7  
8  
9  
10  
11
```

! **TypeError:** num.toUpperCase is not a function

Són els errors més comuns en temps d'execució, causats per descarts de memòria o assumpcions incorrectes sobre les dades.



# Diagnòstic Clínic: Límits i Formats

## RangeError

```
1  
2  
3  
4 new Array(-1);  
5  
6  
7  
8  
9  
10  
11
```

✖ **RangeError:** Invalid array length

## URIError

```
1  
2  
3  
4 decodeURI("%%%");  
5  
6  
7  
8  
9  
10  
11
```

✖ **URIError:** URI malformed



# L'Excepció a la Regla: SyntaxError

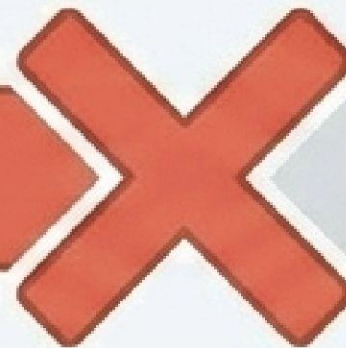


**Els errors de sintaxi NO es poden capturar.**

```
Math.round(4.6; // Falta tancar el parèntesi
```

A diferència dels altres errors, el motor de JavaScript llança el SyntaxError durant la fase d'anàlisi (parsing), abans d'executar el codi. Com que el codi no arriba a arrencar, cap mecanisme de seguretat podrà interceptar-lo.

Parsing



Runtime



# El Circuit de Control d'Execució





# La Mecànica de Captura

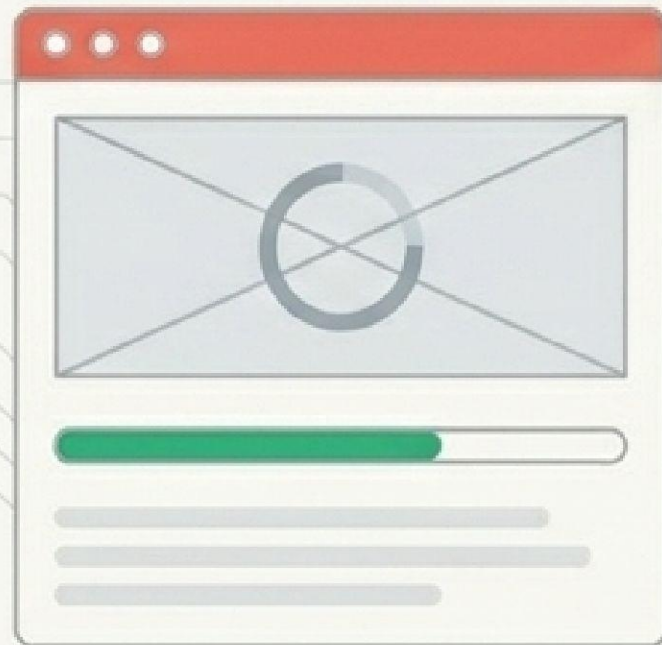
```
try {  
    // Intentem cridar una funció que no existeix  
    funcioInexistent();  
} catch(err) {  
    console.log("Error interceptat: " + err.name);  
}
```



L'aplicació no es penja. El control passa **immediatament** al bloc catch, proporcionant l'objecte err que **conté tota la informació necessària** per registrar o notificar l'incident.

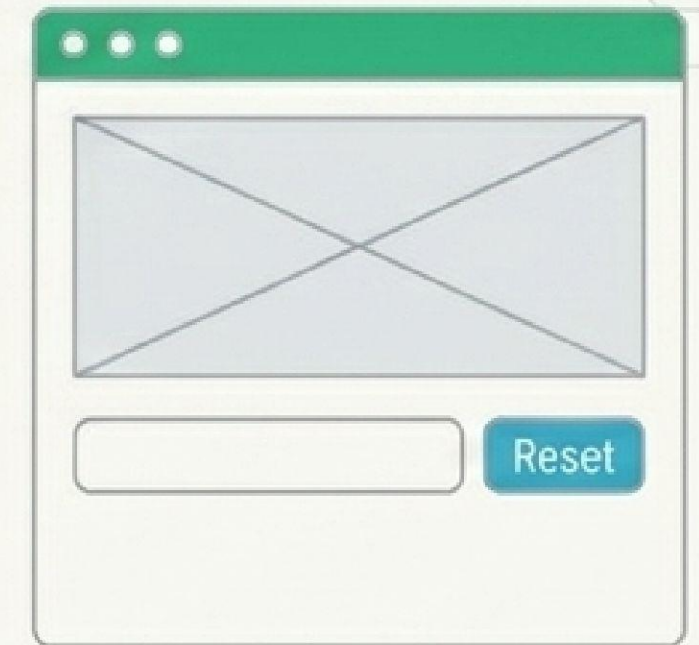


# Neteja Garantida: El bloc finally



Carregant...

```
try {  
    // Operació complexa  
} catch(err) {  
    // Gestió de l'error  
} finally {  
    document.getElementById("demo").value = "";  
    // Aquesta línia s'executa passi el que passi  
}
```



Estat Final: Net

Per què utilitzar **finally** i no posar el codi a sota? Perquè si el bloc **catch** conté un `return`, llança un altre error, o atura l'execució, el codi posterior no s'executaria mai. **finally** assegura la neteja (tancar arxius, amagar *loaders*) de forma garantida.



# Controlant el Flux: Generació d'Errors (throw)

```
throw "Massa gran";
```

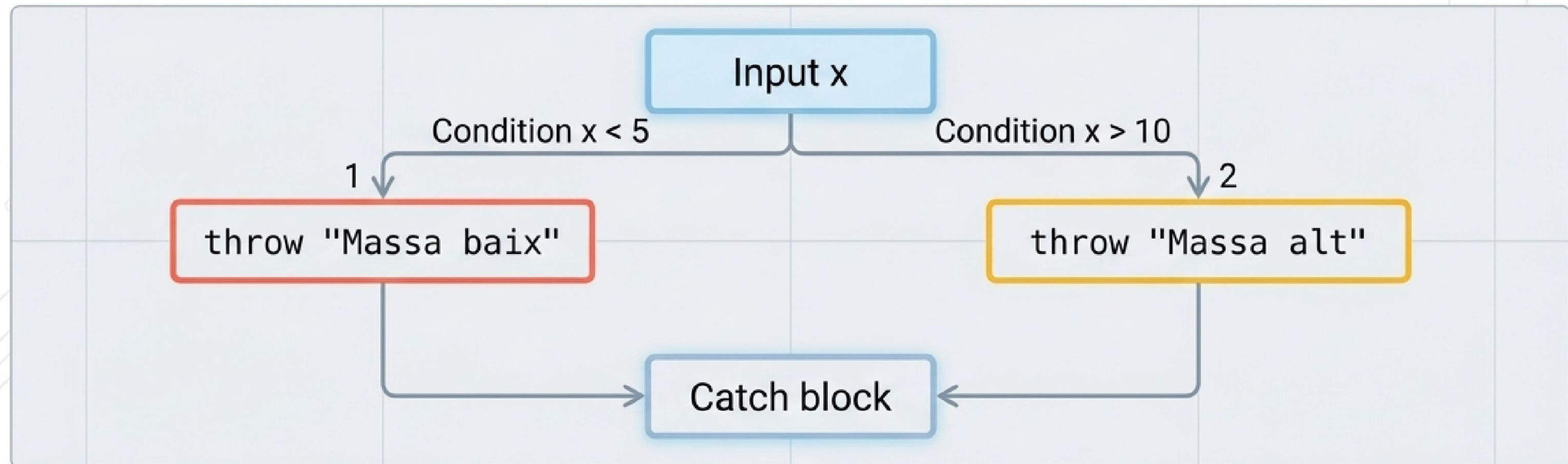
(Text)

```
throw 500;
```

(Número)

```
throw new Error("Dada invàlida");
```

(Objecte - Recomanat)



**Amb throw, pots generar les teves pròpies excepcions per controlar errors de lògica de negoci, no només errors de codi.**



# El Perill Invisible: Els Errors Silenciosos

**El pitjor error no és el que trenca la teva aplicació. És el que permet que l'aplicació continuï funcionant amb dades incorrectes.**

Per raons històriques (les primeres versions de JS no tenien `try...catch`), JavaScript sovint prefereix fallar silenciosament en lloc de llançar excepcions. Aquests errors no aturen l'execució, però causen comportaments imprevisibles.



# Expectativa vs. Realitat: Paranyes Silenciosos

| El Codi                                          | L'Expectativa<br>(Crash/Error) | La Realitat<br>Silenciosa de JS                                      |
|--------------------------------------------------|--------------------------------|----------------------------------------------------------------------|
| <code>parseInt("abc")</code>                     | <code>TypeError</code>         | Retorna <code>NaN</code> (Not-a-Number) i el codi continua.          |
| <code>user.password</code><br>(quan no existeix) | <code>ReferenceError</code>    | Retorna <code>undefined</code> silenciosament.                       |
| <code>if (isActive = true)</code>                | Error de sintaxi o lògica      | Assigna el valor i s'avalua com a cert, executant el bloc per error. |



# La Coerció de Tipus: El Camaleó de JS

## Coerció a Text (+)

"5" + 2

**Resultat: "52"** (Uneix com a text)

## Coerció Numèrica (-, \*, /)

"5" - 2

**Resultat: 3** (Força a número)

⚠️ `5 == "5"` resulta en `true` perquè JS força una conversió abans de comparar.

Això **oculta errors de tipus**. Utilitza la **Igualtat Estricta (===)** per comprovar tant el valor com el tipus alhora.



# Manual de Supervivència: Síntesi de Gestió d'Errors



## PREVENIR

(Contra errors silenciosos)

- Uses `===` sempre (evita coerció).
- Vigila `NaN` en operacions matemàtiques.
- Força tipus explícitament (`Number(x)`).



## CAPTURAR

(Estructura Base)

```
try {  
    // Codi de risc  
} catch (e) {  
    // Xarxa  
} finally {  
    // Neteja  
}
```



## TRACTAR

(Accions)

- Avalua l'Objecte Error (`e.name`, `e.message`).
- Usa `throw` per crear fronteres de dades estrictes.

**⚠ Nota Final:** Recorda, un `SyntaxError` trencarà el procés abans d'arribar aquí.